

CSCI-UA 102 — Quiz 6

Maps and Hash Functions

Name: _____ NetID: _____

Say I create an implementation of `PositionList<Integer>` called `CrazyList`, and I override `equals` so that two `CrazyList` objects are equal if:

1. their first elements are equal,
2. their last elements are equal, and
3. the sum of all their elements are equal.

(1 point) Write two `CrazyLists` of length 4 and 5 that are equal.

Length 4: [_____]

Length 5: [_____]

(4 points) Write an appropriate implementation of `hashCode` for `CrazyList`.

You may assume these methods:

```
public interface Position<E> {  
    E getElement();  
}
```

```
// Methods of PositionList<E>:  
Position<E> first();  
Position<E> last();  
Position<E> after(Position<E> p);
```

Solution

Part 1: Any two lists with the same first element, last element, and sum.

For example: [1, 0, 0, 1] and [1, 0, 0, 0, 1] — both have first = 1, last = 1, sum = 2.

Part 2: Key insight: the `hashCode` contract requires that equal objects produce the same hash code. Since `equals` depends only on `first`, `last`, and `sum`, the hash code must depend only on these three values (not on length or individual elements).

```
public int hashCode() {
    int sum = 0;
    Position<Integer> current = first();
    while (current != null) {
        sum += current.getElement();
        current = after(current);
    }
    int first = first().getElement();
    int last = last().getElement();
    return 31 * (31 * first + last) + sum;
}
```

Note: A simpler `return first + last + sum;` is also correct: it satisfies the contract but produces more collisions. Any formula using only `first`, `last`, and `sum` is valid. Using `length`, individual middle elements, or anything else that equal lists could disagree on would **violate** the contract.